DNA Based Encryption Methods

H. Z. Hsu and R. C. T. Lee
Department of Computer Science
National Chi Nan University
Puli, Nantou Hsieh, Taiwan 545
All correspondences to rctlee@ncnu.edu.tw

Abstract

*In this paper, we shall point out that the DNA sequences possess some interesting properties which we can utilize to encrypt data. We presented three methods, the insertion method, the complementary pair method and the substitution method. For each method, we secretly select a reference DNA sequence **S** and incorporate the secret message **M** into it such that we obtain **S'**. We send this **S'**, together with many other DNA, or DNA-like sequences to the receiver. The receiver is able to identify the particular sequence with **M** hidden in it and ignore all of the other sequences. He will also be able to extract **M**.*

1 Introduction

In recent years, much research work has been done on DNA based encryption schemes [6, 9, 12]. Most of them use biological properties of DNA sequences. The encryption method introduced in this paper does not make use of the biological properties. Instead, we shall use the following properties of DNA sequences.

A DNA sequence is a sequence consisting of four alphabets: A, C, G and T. Each alphabet is related to a nucleotide. It is usually quite long. For instance, the following are two DNA sequences.

The first one is a segment of DNA sequence of Litmus, its real length is with 2856 nucleotides long:
ATCGAATTCGCGCTGAGTCACAATTCGCG
CTGAGTCACAATTCGCGCTGAGTCACAAT
TGTGACTCAGCCGCGAATTCCTGCAGCCC
CGAATTCCGCATTGCAGAGATAATTGTATT
TAAGTGCCTAGCTCGATACAATAAACGCC
ATTTGACCATTCACCACATTGGTGTGCAC
CTCCAAGCTCGCGCACCGTACCGTCTCGA
GGAATTCCTGCAGGATATCTGGATCCACG
AAGCTTCCCATGGTGACGTCACC
[14].

The second one is a segment of DNA sequence of Balsaminaceae, its real length is with 2283 nucleotides long:
TTTTTATTATTTTTTTTCATTTTTTTCTCAG
TTTTTAGCACATATCATTACATTTTATTTTT
TCATTACTTCTATCATTCTATCTATAAAATC
GATTATTTTTATCACTTATTTTTCTAATTTC
CAATATTTCATCTAATGATTATATTACATTA
AAGAAATCGGTTAAAAGCGACTAAAAAT
CAATCTGGAACAAGGCTTAGTTTATTTAAT
ATATTATTTTATGTAATTTCTATTGAAAAAT
TAGTTAAAAGGCAAGTATTTGAGAT [14].

We can now easily see one special property of DNA sequences: There is almost no difference between a real DNA sequence and a faked one. This is a property which we shall exploit in our research.

There is another fact which is quite useful to us: There are a large number of DNA sequences publicly available in various web-sites, such as [14]. A rough estimation would put the number of DNA sequences publicly available to be around 55 million [14].

By using the above facts, we designed three DNA based encryption methods. All of these methods would secretly select a reference sequence *S* from publicly available DNA sequences. Only the sender and the receiver are aware of this reference sequence. The sender would transform this selected DNA sequence *S* into a new sequence *S*' by incorporating the DNA sequence *S* with the secret message *M*. This transformed sequence *S*' is sent by a sender to the receiver together with many other DNA sequences. The receiver would then examine all of the received sequences, identify *S*' and recover the secret message *M*.

We shall introduce three methods in the following sections. For all of these methods, we assume that there are two schemes used by the sender and the receiver which are kept secret. The first one is a binary coding scheme which transforms alphabets A, C, G and T into binary codes and vice versa. For instance, the following may be a binary coding used: ((A 00) (C 01) (G 10) (T 11)). It should be noted that more digits may be used. The second scheme is a complementary pair rule. That is, we shall assign each alphabet x a complement, denoted as $C(x)$. We stipulate that $C(C(x)) \neq x$. The following may be such a rule: ((A C) (C G) (G T) (T A)).

We also assume that the secret message *M* is a binary sequence.

2 Method 1: the Insertion Method

To simplify the discussion, we start with the most basic version and give a simple example. The more complicated version of our method will be presented after this basic one is given. Suppose the secret message *M* is 01001100. Let *S* be ACGGTTCCAATGC. Our coding steps are as follows:

Step 1. We first code *S* into a binary sequence by using the binary coding scheme. Thus the sequence *S* will now become 00011010111101010000111001.

Step 2. Divide *S* into segments whereby each segment contains *k* bits. Suppose *k* is 3. Then we have the following segments: 000, 110, 101, 111, 010, 100, 001, 110, 01.

Step 3. Insert bits from *M*, once at a time, into the beginning of segments of *S*. The result is as follows: 0000, 1110, 0101, 0111, 1010, 1100, 0001, 0110, 01. We should ignore those segments without any secret message inserted. Thus, we will have the following segments: 0000, 1110, 0101, 0111, 1010, 1100, 0001, 0110. Concatenating the above segments, we have the following binary sequence: 00001110010101111010110000010110.

Step 4. We use the binary code scheme to produce the following faked DNA sequence: *S*'=AATGCCCTGGTAACCG. As the reader can see, this sequence is quite different from *S*.

Step 5. We send the above sequence *S*' to the receiver, amid many other irrelevant sequences.

The above process is the encryption process. It is easy to see that the decryption process is just to reverse the encryption process. For every received sequence *T*, the receiver extracts a sequence out of it. If the extracted sequence is not a prefix of the reference sequence *S*, ignore *T*. If it is, the receiver knows that he has also successfully extracted the secret message *M* as a by-product.

The above is the basic version of our approach. In a more complicated version, we divide *S* into segments by using a random number generator. That is, *k* is not fixed any more. Instead, it is determined by a random number generator which is known only to the sender and the receiver. Suppose the *k*'s are 6, 3, 2, 4. Then *S* is divided into segments with lengths 6, 3, 2 and 4 respectively. Note that there is also a secret message *M*. We may therefore also use the same random number generator to divide *M* into segments. The parameter used will be denoted as *r*.

The following is the exact algorithm for Method 1.

Algorithm 2-1 Encryption Algorithm of Method 1 (Insertion Method)

Input: A reference DNA sequence *S*, a secret binary message *M* and a binary coding scheme to code A, C, G and T into binary digits.

Output: An encrypted DNA sequence *S*'.

Step 1. Code *S* into a binary sequence S_1 by using the binary coding scheme.

Step 2. Generate *k*'s by using a random number generator to divide S_1 into segments and generate *r*'s to divide the secret message *M* into segments. Each k_i and r_i is larger than 1 or equal to 1. Denote S_1 by $s_1 s_2 \cdots s_n$ and *M* by $m_1 m_2 \cdots m_p$.

Step 3. nsert each m_i of *M* before s_i of S_1 to produce a new binary sequence. Delete $s_{p+1} s_{p+2} \cdots s_n$. Denote the resulting binary sequence by S_2.

Step 4. Transform sequence S_2 back to a faked DNA sequence S_3 by using the same binary coding scheme used in

Step 1.

Step 5. Return S_3.

S_3 is sent to the receiver together with many other DNA sequences. The receiver uses the following algorithm to decrypt.

Algorithm 2-2 Decryption Algorithm for Method 1

Input: A set of DNA sequences, one of which has the secret message M hidden in it by using Algorithm 2.1, a reference DNA sequence S and a binary coding scheme.

Output: The secret message M.

Step 1. Generate numbers k's and r's denoted as $k_1k_2...k_n$ and $r_1r_2...r_p$ by using the same random number generator with the same seed of the encoding scheme.

Step 2. For a DNA sequence S' of the set, code S' into a binary sequence by using the binary coding used by the sender and use $r_1+k_1, r_2+k_2,...$ to divide the binary sequence into binary segments.

Step 3. For each segment of the first p segments of S', extract the first r_i bits, called m_i.

Step 4. For each segment of the first p segments of S', extract the last k_i bits, called s_i.

Step 5. Concatenate all m_i's to be M and all s_j's, to be S_1.

Step 6. Transform S_1 to be a DNA sequence by using the same rule. If S_1 is not a prefix of S, go to **Step 2**.

Step 7. Return M.

For an intruder to find out the secret message, he must be equipped as follows. (1) He must know precisely the reference DNA sequence S. Since there are roughly 55 millions DNA sequences available publicly, it is extremely hard to guess one. (2) He has to know the random number generator and the two seeds used. (3) He has to know the binary coding scheme.

3 Method 2: the Complementary Pair Approach

In this section, we shall illustrate Method 2, the complementary pair approach. In RNA sequence, we often have the so-called base pairs [4, 8, 13]. We cannot elaborate the detailed meaning of the base pairs of RNA in this paper. For us, we may just define our own complementary pairs. That is, for each alphabet, we assign a unique counterpart for it. For instance, we may have the following base pair rule:

(A T) (C A) (G C) (T G).

as discussed in Section 1. Then the complementary sequence of AATGC will be TTGCA.

In the sequence: "ATCTG**AATGC**TTGTCTA**TTGCA**TCAAT", complementary substrings occur, as indicated by the bold characters. To find the longest complementary substrings, we may use dynamic programming approach [1]. Let us assume that we have the secret message M which has even number of bits. Note that it is always reasonable to assume so because we may always use even number of bits to code. To give an example, assume that M=0110. Again, as in Method 1, we assume that we have a reference sequence S. Let us assume it to be S=ACGGTCGTTCCCTAGTTG. Our Method 2 will work as follows:

Step 1. Artificially generate a sequence without any complementary substrings and consisting of A, C, G and T only. Assume that the sequence is L=ACGGTCTCATCAATGCTTCAGT.

Step 2. Divide M into segments such that each segment contains even number of bits. Thus, in our case, we have 01 and 10. We code 01 and 10 according to some coding rule. By using the coding rule given in the previous section, 01 and 10 will be coded as C and G respectively.

Step 3. Generate two complementary strings with length k and insert them into L. Assume k=5 and we have the following two complementary strings: (AAGCT TTCAG) and (ACCTG TAAGC). The sequence L now becomes L'= ACG **AAGCT** GTCT **TTCAG** CAT **ACCTG** CAAT **TAAGC** GCTTCAGT.

Step 4. Insert the first(second) alphabet of the secret message one alphabet before the first(second) complementary string. The string becomes: L'= AC**C**G **AAGCT** GTCT **TTCAG** CA**G**T **ACCTG** CAAT **TAAGC** GCTTCAGT.

Step 5. Use a random number generator to select two positive integers j and i. Assume j=2 and i=4. Insert substrings

$S[j,j+i]$=CGGTC and $S[2j,2j+i]$=GTCTC one alphabet after the first and second complementary substrings. L' becomes: L"=AC**C**G**A** **AGCT**GTCT**TTCAG**C**CGGTC**A**G**T **ACCTG**CAAT**TAAGC**G**GTCTC**CTT CAGT.

Step 6. Return L''.

The sender sends L" together with many other sequences, some with complementary pair substrings, to the receiver.

The receiver would process every sequence received by finding all complementary substrings of it. If they are of the correct lengths, then it checks whether the prescribed substrings of S are hidden correctly. If they are, the receiver may now extract the secret message.

The above description of Method 2 is a simplified version. The more complicated version may use complementary pair substrings with varying lengths. As we indicated above, the sender sends L'' together with many other sequences which may contain complementary pair substrings. Those complementary pair substrings will have lengths different from the specified ones and thus will be ignored.

Instead of complementary pair substrings, we may also use palindromes [2, 3, 5, 7, 10, 11]. A palindrome is a string of the form $\alpha\alpha'$ where α' is the reverse of α. For instance: AACGTTGCAA is a palindrome where α =AACGT. There are several methods proposed to find the longest palindrome in a string [2, 3, 5, 7, 10].

Our approach using palindrome is quite similar to that using complementary pairs. Let $\alpha = \alpha_1\alpha_2...\alpha_h$ be a substring. Let $\alpha_h' = C(\alpha_h)$. Let $\alpha' = \alpha_h'\alpha_{h-1}'..\alpha_1'$. Then $\alpha\alpha'$ is a complementary palindrome. For example, assume the complementary rule is ((AT) (CA) (GC) (TG)). Then ACCTGAAT is a complementary palindrome.

Method 2 may use complementary palindromes because methods to find palindromes can be easily extended to find the complementary palindromes.

In the following, we shall present the algorithms for Method 2. The algorithms are based upon the complementary pair concept.

Algorithm 3.1 Encryption Algorithm for Method 2 (Complementary Pair Approach)

Input: A reference DNA sequence S, a secret binary message M, a binary coding scheme to code A, C, G and T into binary digits and a complementary pair rule.

Output: An encrypted DNA sequence S'

Step 1. Artificially generate a DNA-like sequence $L=l_1l_2\ldots l_s$ with length s and without any complementary substrings. Use a random number generator to generate $k_1, k_2, \cdots, k_n$. Generate a set $A=\{a_1a_1', a_2a_2', \ldots, a_na_n'\}$, $n<<s$ and each a_ia_i' with length k_i, of complementary strings.

Step 2. Divide M into segments such that each segment may be coded by the binary coding scheme. Code these segments to be "A", "G", "C" and "T" by using the same coding rule. Suppose $M=m_1m_2\ldots m_p$.

Step 3. For each a_g and a_g', insert them after l_{g+3}, and l_{g+k+3}, $1 \le g \le p$.

Step 4. For each pair of complementary strings a_g and a_g' in L, insert m_g one alphabet before a_g.
Thus, L will be a new sequence, called L'.

Step 5. Randomly select two positive integers j and i. For each a_g' in L', insert $S[jg,jg+i]$ one alphabet after a_g'. Thus, L will be a new sequence, called L".

Step 6. Return L".

Algorithm 3.2 Decryption Algorithm of Method 2 (Complementary Pair Approach)

Input: A set of DNA sequences, one of which has the secret message M hidden in it by using Algorithm 3.1, a reference DNA sequence S, a binary coding scheme and a complementary pair rule used in Algorithm 3.1.

Output: Secret message M.

Step 1. For the next DNA sequence L" in the set, use the dynamic programming strategy to find out all complementary substrings. If the substrings are not of the correct lengths or no such strings are found, go back to **Step 1**.

Step 2. Select two positive integers j and i by using the same random number generator used in Algorithm 3.1. For each pair of complementary substrings a_g and a_g', check whether the substring with length i-1 starting from one alphabet after a_g' is the same with $S[jg,jg+i]$ or not. If they are not the same, ignore L" and go to **Step 1**.

Step 3. For each pair of complementary substrings a_g and a_g', extract the alphabet one alphabet before a_g, called m_g.

Step 4. Concatenate all m_g's to be M.

Step 5. Return M.

An intruder should have the following information. First, he should know the reference DNA sequence. Second, he should have the complementary rule. Third, he should know the positive integers j and i to make an authentication to the reference DNA sequence. Fourth, he should know the binary coding rule. Fifth, he must know the correct lengths of the complementary pair substrings.

4 Method 3: the Substitution Approach

For this method, we also will use a reference sequence S. Let us assume that S=ACGGAATTGCTTCAG and the secret message $M=m_1m_2\ldots m_p$ is 0111010. The length of S is 15 and is larger than the length of M, p, which is 7 in this case. For illustration, assume that the complementary rule is the same as given in the above sections. That is, the rule is as follows: ((A T) (C A) (G C) (T G)).

Our main idea is as follows:

Step 1. Suppose the length of the reference sequence S is 15. Select p distinct numbers randomly from 1 to 15, p is equal to 7 in this case. Assume that they are sorted as 2, 3, 5, 10, 12, 13 and 15. Let $A=A_1, A_2, \ldots, A_p$={2,3,5,10,12,13,15}.

Step 2. Transform S into S' by the following rule:
For all i from 1 to 15,
if i is equal to some A_j and m_j is 1, $1 \le j \le p$, set S_i to $C(S_i)$,
if i is equal to some A_j and m_j is 0, do not change S_i, and
if i is not equal to any A_j, set S_i to $C(C(S_i))$.
Thus S'=GCCATGCCAACTAGG.

Step 3. Send S' to the receiver with many other sequences.

The receiver would not need to generate the set A. After receiving a set of sequences, he will check all positions of each sequence S' in the set. There are only three possible cases: (1) S'_i is the same with S_i (The secret bit m_j is equal to 0). (2) S'_i is $C(S_i)$ (The secret bit m_j is equal to 1). (3) S'_i is $C(C(S_i))$. If there exists one j such that S'_j and S_j are not of the above three cases, it means that the sequence should be ignored.

In the following, we present the algorithms for Method 3.

Algorithm 4-1 Encryption Algorithm for Method 3 (Substitution Approach)

Input: A DNA sequence S, the secret message $M=m_1m_2\ldots m_p$ and a complementary rule.

Output: An encrypted DNA sequence S".

Step 1. Use a random number generator to generate a set of available integer sequences, called set A. The number of A is p, the length of M.

Step 2. Initialize i and j to 1.

Step 3. For each element S_i of S, do the following operations:
if i is equal to some A_j and m_j is 1, $1 \le j \le p$, change S_i to $C(S_i)$,
if i is equal to some A_j and m_j is 0, do not change S_i,
if i is not equal to any A_j, set S_i to the double-complement of itself.1

Step 4. Return S'.

Algorithm 4-2 Decryption Algorithm for Method 3 (Substitution Approach)

Input: A set of DNA sequences, the reference sequence S and the complementary rule.

Output: Secret message M.

Step 1. Initialize i and j equal to 1.

Step 2. For the next sequence S' of the sequence set, do the following operations:
For each S_i:
if there exists a j such that such that $S_j' \ne S_j, S_j' \ne C(S_j')$ and $S_j' \ne C(C(S_j))$, ignore S'; otherwise,
if S'_i is the same with S_i, setm_j=0 and increment j,
else if S'_i is the same with $C(S_i)$, set m_j=1 and increment j.

Step 3. Concatenate all m_j,'s to be M and return M.

For an intruder to find out the secret message, he should have the following information. (1) He should know the reference DNA sequence. (2) He should know the complementary rule.

5 Concluding Remarks

In this paper, we have pointed out that the DNA sequences have the special properties which we can utilize for encryption purposes. We have given three methods and they are all based upon a reference sequence known only to the sender and the receiver. This reference sequence can be selected from any web-site associated with DNA sequences. Since there are many web-sites and roughly 55 million publicly available DNA sequences, it is virtually impossible to guess this sequence.

For further research, we shall investigate some mathematical properties of our approaches.

References

[1] Akutsu, T., Dynamic Programming Algorithms for RNA Secondary Structure Prediction with Pseudoknots, Dirsrete Applied Mathematics, Vol. 104, 2000, pp. 45-62.

[2] Apostolico, A., Breslauer, D., Galil, Z., Optimal Parallel Algorithms for Periods, Palindromes and squares, Proceedings of the International Colloquium on Automata, Languages, and Programming , 1992 , pp. 296-307 .

[3] Apostolico, A., Breslauer, D., and Galil, Z., Parallel Detection of All palindromes in a String, Theoretical Computer Science , Vol. 141 , 1995 , pp. 163-173 .

[4] Alberts, B., Bray, D., Lewis, J., Raff, M., Roberts, K. and Watson, J. D., Molecular Biology of the Cell, New York & London: Garland Publishing, 1994.

[5] Breslauer, D., and Galil, Z., Finding All Periods and Initial Palindromes of a String, Algorithmica , Vol. 14 , 1995 , pp. 355-366 .

[6] Clelland, C. T., Risca, V. and Bancroft, C., Hiding Messages in DNA Microdots, Nature, Vol. 399, 1999, pp.533-534.

[7] Gail, Z., Real-Time Algorithms for String-Matching and Palindrome Recognition, STOC , 1976 , pp. 161-173 .

[8] Lehninger, A. L., Nelson, D. L. and Cox, M. M., Principles of Biochemistry, New York, Worth, 2000.

[9] Leier, A., Richter, C., Banzhaf, W. and Rauhe, H., Cryptography with DNA Binary Strands, BioSystems, Vol. 57, 2000, pp.13-22.

[10] Manacher, G.., A new linear-time on-line Algorithm for finding the smallest initial palindrome of the string, J. ACM, Vol. 22, 1975, pp. 346-351.

[11] Crochemore, M. and Rytter, W., Jewels of Stringology, World Scientific, 2002.

[12] Shimanovsky, B., Feng, J. and Potkonjak, M., Hiding Data in DNA, Revised Papers from the 5th International Workshop on Information Hiding, Lecture Notes in Computer Science, Vol. 2578, 2002, pp.373-386.

[13] Watson, J., Hopkins, N., Roberts, J. and Steitz, J., Molecular Biology of the Gene, Benjamin Cummings, 4th Edition, Menlo Park, CA, 1987.

[14] European Bioinformatics Institute, URL: http://www.ebi.ac.uk/.